



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE



Metodologias ágeis: Scrum, Kanban e a escolha do processo

Introdução ao Agile com Scrum, Lean e
Kanban



O Que Esta Aula Precisa Responder

Para o nosso projeto com stakeholder real, qual cadência faz mais sentido?

Essa pergunta não é retórica. Ao final desta aula, cada equipe deve ter uma resposta clara — e essa resposta vai guiar a condução do projeto até o final do semestre. O objetivo não é apresentar os frameworks do zero: vocês já leram. É forçar uma **decisão fundamentada** e criar consciência sobre os trade-offs que ela carrega.

Não é uma guerra ideológica

Scrum e Kanban têm contextos legítimos. Os critérios de escolha importam mais do que a preferência pessoal.

É uma decisão real

A escolha de hoje define como será a sprint planning de quinta-feira e o ritmo das próximas 8 semanas.

Tem consequências

Um processo escolhido sem compreensão real não protege ninguém — só cria rituais vazios.



Para o projeto de vocês —> Scrum,
Kanban ou outro? Quem quer
começar?



Introdução às Metodologias Ágeis

Origem e Filosofia

As metodologias ágeis surgiram como resposta às limitações dos modelos tradicionais. Baseadas no Manifesto Ágil de 2001, essas abordagens priorizam indivíduos e interações, software funcional, colaboração com o cliente e resposta a mudanças. O foco está na entrega rápida de valor ao cliente através de ciclos curtos de desenvolvimento.

Principais Características

- Desenvolvimento iterativo e incremental
- Comunicação contínua com o cliente
- Equipes auto-organizadas e multifuncionais
- Adaptação rápida a mudanças
- Ênfase na qualidade e melhoria contínua



Scrum: Framework Ágil Popular



O Scrum é um framework ágil que enfatiza a colaboração, flexibilidade e entrega de valor em intervalos regulares.

Ele define papéis claros (Product Owner, Scrum Master, Development Team) e eventos específicos para garantir

transparência e adaptabilidade ao longo do projeto.



Agile lifecycle

- Aceita as mudanças como um fato da vida: melhoria contínua vs. fases
- Os desenvolvedores refinam continuamente o protótipo em funcionamento, mas incompleto, até que os clientes fiquem satisfeitos, com feedback do cliente a cada **iterações** (1-2 semanas)
- **Todos os elementos do ciclo de vida em todas as iterações**
- O Agile enfatiza o **Test-Driven Development (TDD)** para reduzir erros, especificando **User Stories** para validar os requisitos do cliente, **Velocity** para medir o progresso



Velocidade de um time

Número de story points que o time consegue implementar em um sprint

Definição de story points é "empírica"



Story points

Unidade (inteiro) para comparação do tamanho de histórias.

Exemplo de escala: 1, 2, 3, 5, 8, 13

História	Story Points
Cadastrar usuário	8
Postar perguntas	5
Postar respostas	3
Tela de abertura	5
Gamificar perguntas e respostas	5
Pesquisar perguntas e respostas	8
Adicionar tags em perguntas e respostas	5
Comentar perguntas e respostas	3

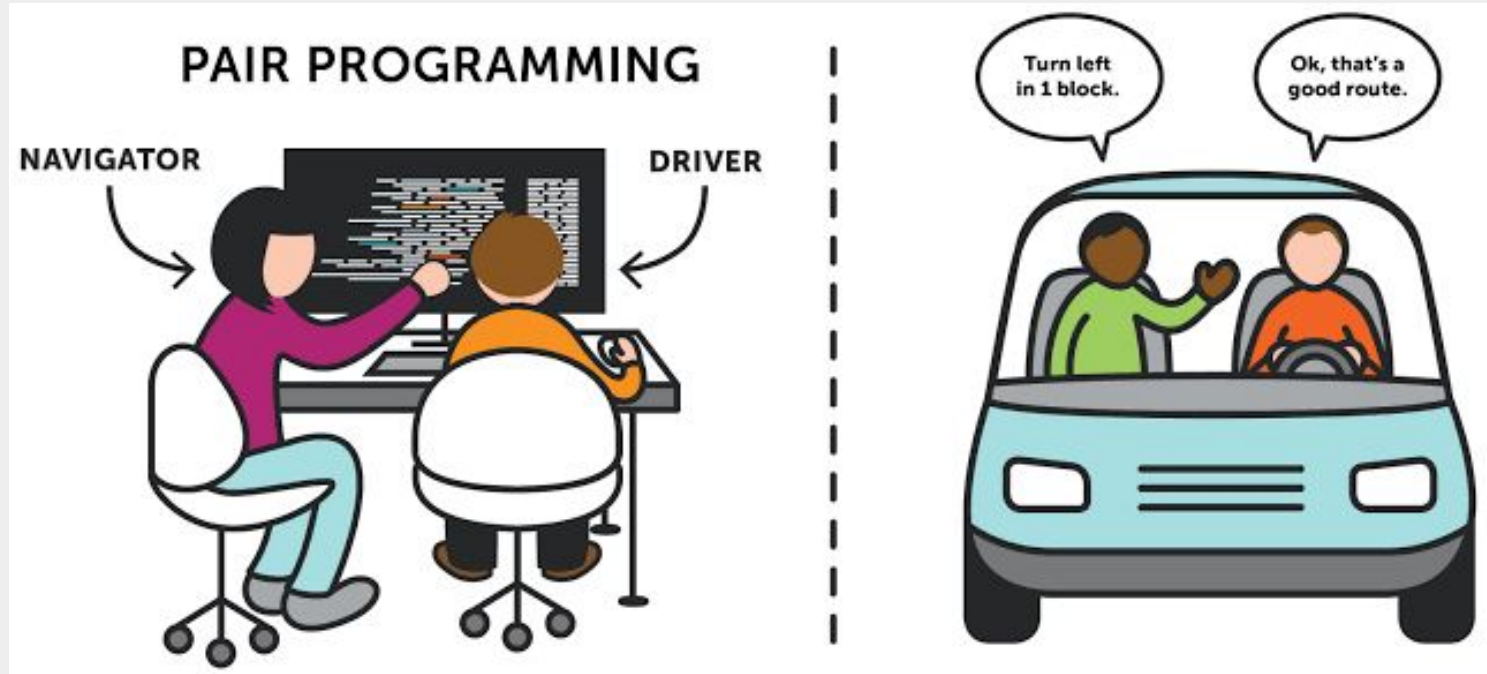


“Extreme Programming” (XP), uma versão do ciclo de vida ágil (Kent Beck et al.)

- If **short iterations are good**, make them **as short as possible** (weeks vs. years)
- If **simplicity is good**, always do the **simplest thing** that could possibly work
- If **testing is good**, test all the time. Write the test **just before** writing the code to be tested.
- If **code reviews are good**, **review code continuously**, by **programming in pairs**, taking turns looking over each others' shoulders.

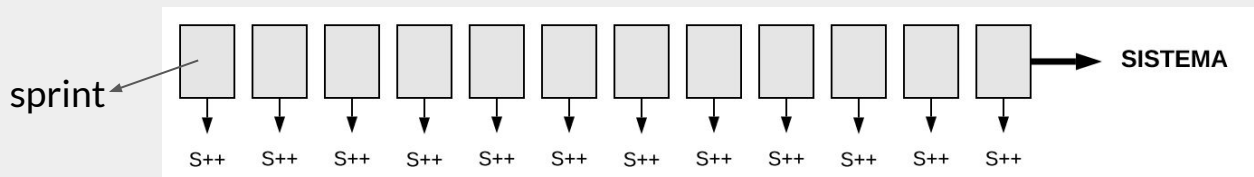


Pair Programming



Scrum vs XP

- Proposto por Jeffrey Sutherland e Ken Schwaber (OOPSLA 1995)
- Scrum não é apenas para projetos de software
 - Logo, não define práticas de programação, como XP
- Scrum define um "processo" mais rígido que XP
 - Eventos, papéis e artefatos bem claros
 - Como em qualquer método ágil, desenvolvimento é dividido em sprints (iterações)
 - Duração de um sprint: até 1 mês, normalmente 15 dias



O que se faz em uma sprint?

- Implementa-se algumas **histórias dos usuários**
- Histórias = funcionalidades (ou features) do sistema
- Exemplo: fórum de perguntas e respostas

Postar Pergunta

Um usuário, quando logado no sistema, deve ser capaz de postar perguntas. Como é um site sobre programação, as perguntas podem incluir blocos de código, os quais devem ser apresentados com um layout diferenciado.

Bem simples, deve caber em um post-it



Quem escreve as histórias?

- Product Owner (PO)
 - Membro (papel) obrigatório em times Scrum
 - Especialista no domínio do sistema
- Suas funções?
 - Escrever histórias dos usuários
 - Explicar histórias para os devs, durante o sprint
 - Definir "testes de aceitação" de histórias
 - Priorizar histórias
 - Manter o backlog do produto



Fonte: <https://annelisegripp.com.br/rotina-de-um-product-owner/>



Backlog do Produto

- Lista de histórias do usuário, que foram escritas pelo PO
- Duas características:
 - Priorizada: histórias do topo têm maior prioridade
 - Dinâmica: histórias podem sair e entrar, à medida que o sistema evolui
- Quais histórias vão entrar no próximo sprint?
 - Essa decisão é tomada no início do sprint
 - Em uma reunião chamada de planejamento do sprint
 - PO propõe histórias que gostaria de ver implementadas
 - Devs decidem se têm velocidade para implementá-las



Resumindo Scrum

- Elementos principais
 - Sprint (evento)
 - PO e Devs (papeis)
 - Backlog do produto (artefato)
- Importante!
 - Em um time scrum, todos têm o mesmo nível hierárquico
 - PO não é o "chefe" dos Devs
 - Devs têm autonomia para dizer que não vão conseguir implementar tudo que o PO quer em um único sprint



Scrum: Além da Definição do Livro

O Sprint é um **compromisso de entrega**, não de tentativa. Ao final, o time tem algo funcionando para mostrar ao stakeholder.

Elementos que mais confundem

Sprint Planning realista: subestimar o esforço tem custo real na revisão.

Backlog refinado: itens vagos geram sprints sem entrega.

Velocity: não é meta — é ferramenta de aprendizado ao longo do tempo.

Os três papéis no time de vocês: quem é o guardião do backlog? Quem garante que o processo está funcionando?



Kanban: Muito Além do Quadro de Post-its



Princípio Central: Fluxo

Kanban limita o **trabalho em progresso (WIP)** em vez de limitar o tempo. O foco é remover bloqueios e otimizar a entrega contínua — não encaixar tudo numa caixa de tempo.



Métricas: Cycle Time e Throughput

Sem sprint, não há velocity. O Kanban mede quanto tempo um item leva do início ao fim (*cycle time*) e quantos itens são concluídos por período (*throughput*).



Melhor Para Demandas Imprevisíveis

Kanban é mais adequado quando o escopo muda constantemente — suporte, manutenção, operações. Projetos com escopo e prazo definidos pedem outra abordagem.



Kanban: Fluxo Contínuo e Visualização

Princípios do Kanban

- Visualize o fluxo de trabalho
- Limite o trabalho em progresso (WIP)
- Gerencie e otimize o fluxo
- Torne as políticas explícitas
- Implemente loops de feedback
- Melhore colaborativamente e evolua experimentalmente

Benefícios do Kanban

O Kanban oferece flexibilidade, transparência e foco na entrega contínua. Ele ajuda a identificar gargalos, reduzir desperdícios e melhorar a eficiência do processo. A visualização clara do trabalho promove uma melhor colaboração e compreensão do fluxo de valor entre todos os membros da equipe.



Scrum vs. Kanban: Quadro Comparativo

Critério	Scrum	Kanban
Tipo de demanda		
Cadência		
Planejamento		
Métricas		
Stakeholder		
Melhor para...		



Scrum vs. Kanban: Quadro Comparativo

Critério	Scrum	Kanban
Tipo de demanda	Escopo definido, entregas planejadas	Demanda contínua, variável e imprevisível
Cadência	Sprints fixos (1-4 semanas)	Fluxo contínuo, sem caixas de tempo
Planejamento	Sprint planning com comprometimento	Just-in-time, conforme a demanda chega
Métricas	Velocity, burn-down chart	Cycle time, throughput, WIP
Stakeholder	Revisão ao final de cada sprint	Entrega contínua de itens concluídos
Melhor para...	Projetos com escopo e prazo definidos	Suporte, manutenção, operações contínuas



Lean: A Filosofia Que Atravessa Tudo

O Lean não é um processo — é uma filosofia. Seu princípio central: eliminar desperdício.

No contexto de software, desperdício assume formas concretas e muitas vezes invisíveis:



Funcionalidades Não Usadas

Código desenvolvido sem validação do stakeholder, esforço que nunca gera valor.



Reuniões Sem Propósito

Encontros sem pauta clara, sem decisão tomada, sem próximo passo definido.



Retrabalho Evitável

Código que nunca vai para produção ou que precisa ser refeito por falta de alinhamento inicial.



No Projeto de Vocês

Quais são os desperdícios mais prováveis nos próximos dois meses? Identifique agora, antes que aconteçam.



Lean: Eliminando Desperdício

Identificar Valor

Compreender o que realmente agrega valor ao cliente e ao produto final.

Mapear Fluxo de Valor

Visualizar todas as etapas do processo para identificar e eliminar atividades que não agregam valor.

Criar Fluxo

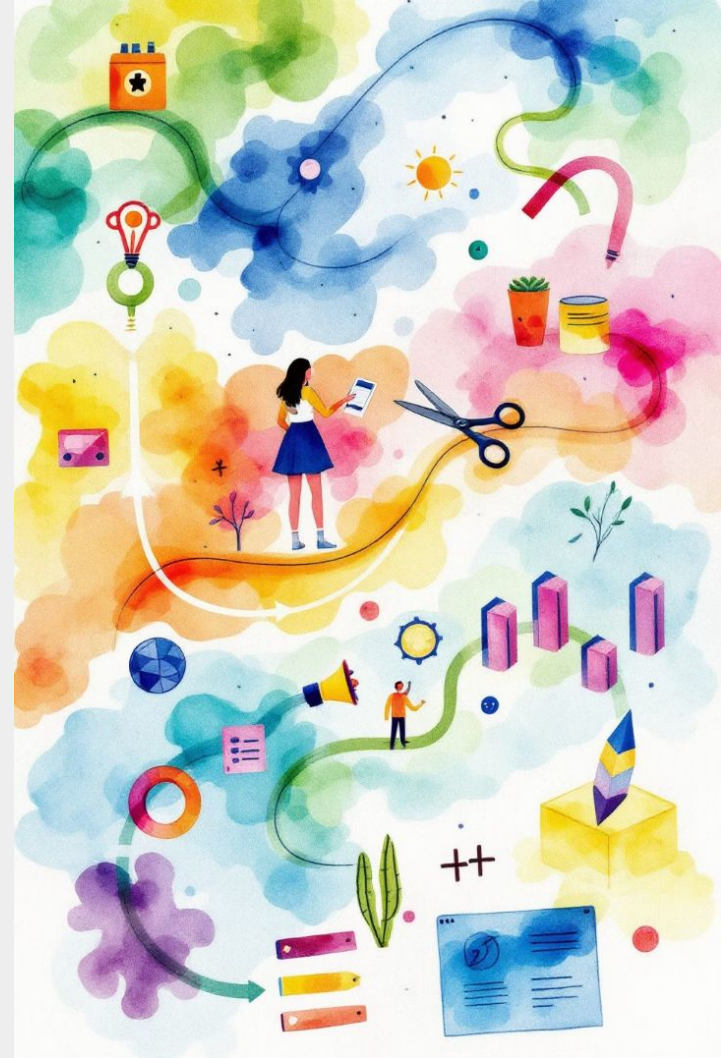
Garantir que as etapas que agregam valor fluam suavemente, sem interrupções ou atrasos.

Estabelecer Pull

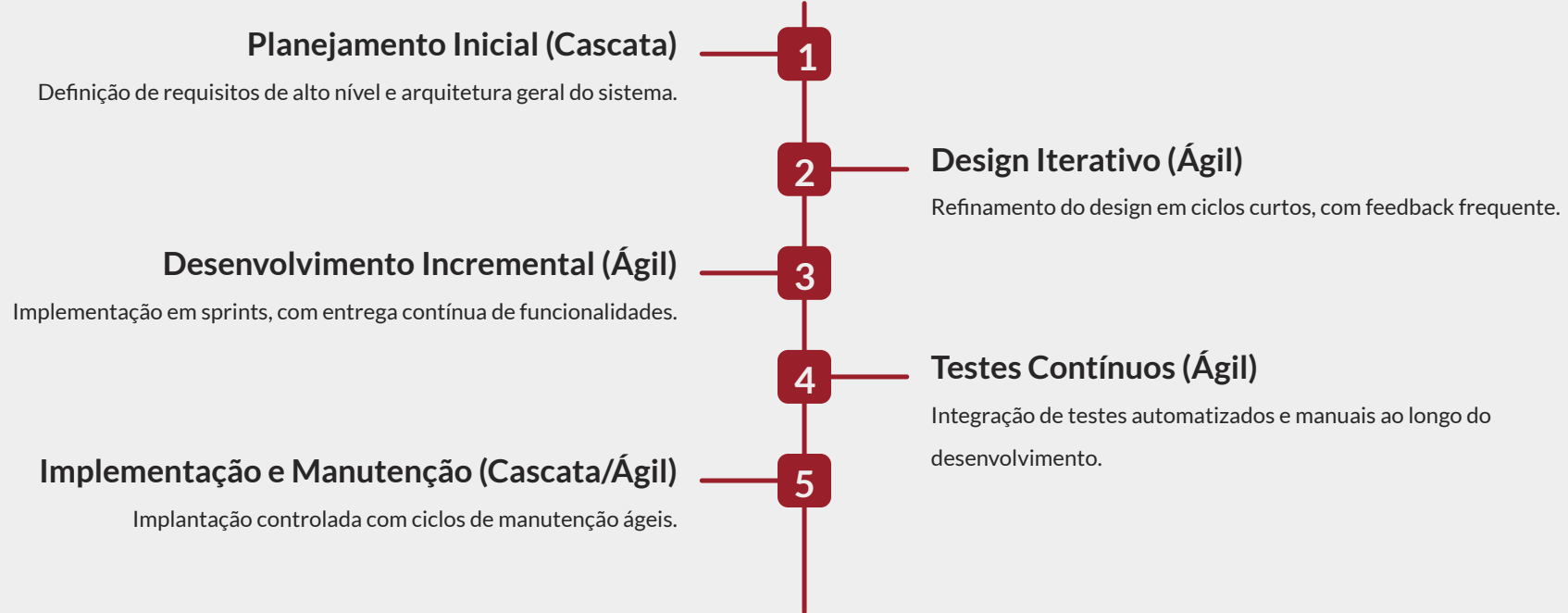
Produzir apenas o que é necessário, quando é necessário, reduzindo o trabalho em andamento.

A abordagem Lean, originária da manufatura, foi adaptada para o desenvolvimento de software com foco na maximização do valor e minimização do desperdício.

Promove a melhoria contínua e a eficiência em todos os aspectos do processo de desenvolvimento.



Modelos Híbridos: O Melhor dos Dois Mundos



Modelos híbridos combinam elementos de abordagens tradicionais e ágeis, adaptando-se às necessidades específicas do projeto e da organização. Esta flexibilidade permite aproveitar a estrutura do modelo cascata para planejamento de longo prazo, enquanto incorpora a agilidade necessária para responder rapidamente às mudanças.



Critérios de Comparação entre Modelos



Complexidade do Projeto

Avalie se o projeto é simples e bem definido ou complexo e sujeito a mudanças frequentes.



Tempo de Desenvolvimento

Considere se o projeto tem um prazo fixo ou pode beneficiar-se de entregas incrementais.



Tamanho e Experiência da Equipe

Analise as habilidades e a experiência da equipe com diferentes metodologias.



Envolvimento do Cliente

Determine o nível de participação e feedback do cliente esperado durante o desenvolvimento.

Ao comparar modelos de processo, é crucial considerar múltiplos fatores para escolher a abordagem mais adequada ao contexto específico do projeto e da organização.



Flexibilidade vs. Estabilidade dos Requisitos

Requisitos Estáveis

Para projetos com requisitos bem definidos e pouca probabilidade de mudança, modelos mais estruturados como o Cascata ou V-Model podem ser apropriados. Esses modelos oferecem uma abordagem linear e previsível, ideal para sistemas críticos ou regulamentados.

Requisitos Flexíveis

Projetos com requisitos em evolução ou incertos se beneficiam de metodologias ágeis como Scrum ou Kanban. Estas abordagens permitem adaptações rápidas, entregas frequentes e incorporação contínua de feedback, essenciais em ambientes dinâmicos de negócios.



Escolhendo o Modelo Adequado

1

Análise do Projeto

Avalie escopo, complexidade e riscos.

2

Contexto Organizacional

Considere cultura, recursos e experiência da equipe.

3

Expectativas do Cliente

Alinhe com as necessidades e envolvimento do cliente.

4

Seleção do Modelo

Escolha baseada na análise dos fatores anteriores.

A escolha do modelo de processo adequado é crucial para o sucesso do projeto. É importante considerar cuidadosamente todos os aspectos do projeto, da organização e do cliente antes de tomar uma decisão. Lembre-se de que a flexibilidade para adaptar o modelo escolhido às necessidades específicas do projeto também é fundamental.



Exemplos Práticos de Aplicação



Sistema Bancário

Modelo Cascata ou V-Model devido à necessidade de segurança, conformidade regulatória e requisitos bem definidos.

Aplicativo Móvel

Metodologia Ágil (Scrum ou Kanban) para rápida iteração, feedback do usuário e adaptação às mudanças do mercado.

Sistema de Saúde

Abordagem Híbrida, combinando a estrutura do Cascata para componentes críticos com agilidade para funcionalidades não críticas.

E-commerce

Desenvolvimento Incremental ou Ágil para lançamento rápido de MVPs e evolução contínua baseada em métricas de usuário.



Conclusão e Lições Aprendidas

Principais Takeaways

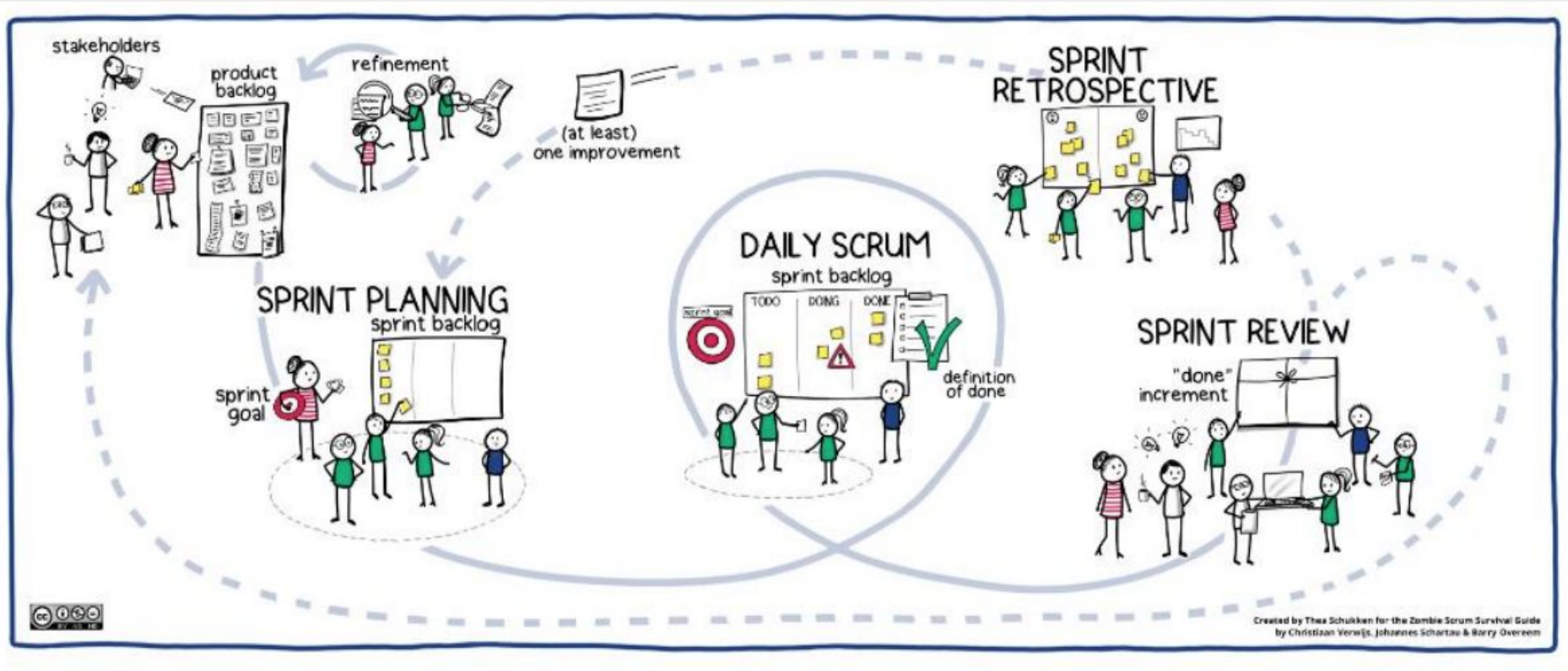
- Não existe um modelo "único para todos" - a escolha depende do contexto
- Flexibilidade e adaptabilidade são cruciais no ambiente de desenvolvimento moderno
- O envolvimento contínuo do cliente e da equipe é fundamental para o sucesso do projeto
- A melhoria contínua do processo é tão importante quanto o produto final

Olhando para o Futuro

À medida que a tecnologia e as necessidades de negócios evoluem, os modelos de processo de software continuarão a se adaptar. A chave para o sucesso será a capacidade de combinar metodologias, ferramentas e práticas de forma eficaz, sempre com foco na entrega de valor ao cliente e na qualidade do software.



Visão geral



Resumo Visual: Os Três Frameworks

Uma visão consolidada dos três frameworks discutidos na aula — seus princípios centrais, métricas e melhor contexto de aplicação.



Scrum

Princípio: Comprometimento de entrega em ciclos curtos (sprints).
Papéis definidos, cerimônias estruturadas, melhoria contínua via retrospectiva.

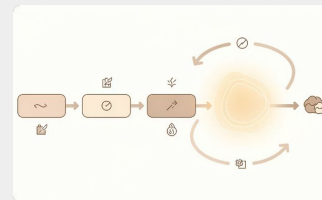
Melhor para: Projetos com escopo definido e revisões periódicas com stakeholder.



Kanban

Princípio: Limitar o trabalho em progresso (WIP) para otimizar o fluxo.
Sem sprints — entrega contínua de itens concluídos.

Melhor para: Demandas contínuas, suporte, manutenção e operações imprevisíveis.



Lean

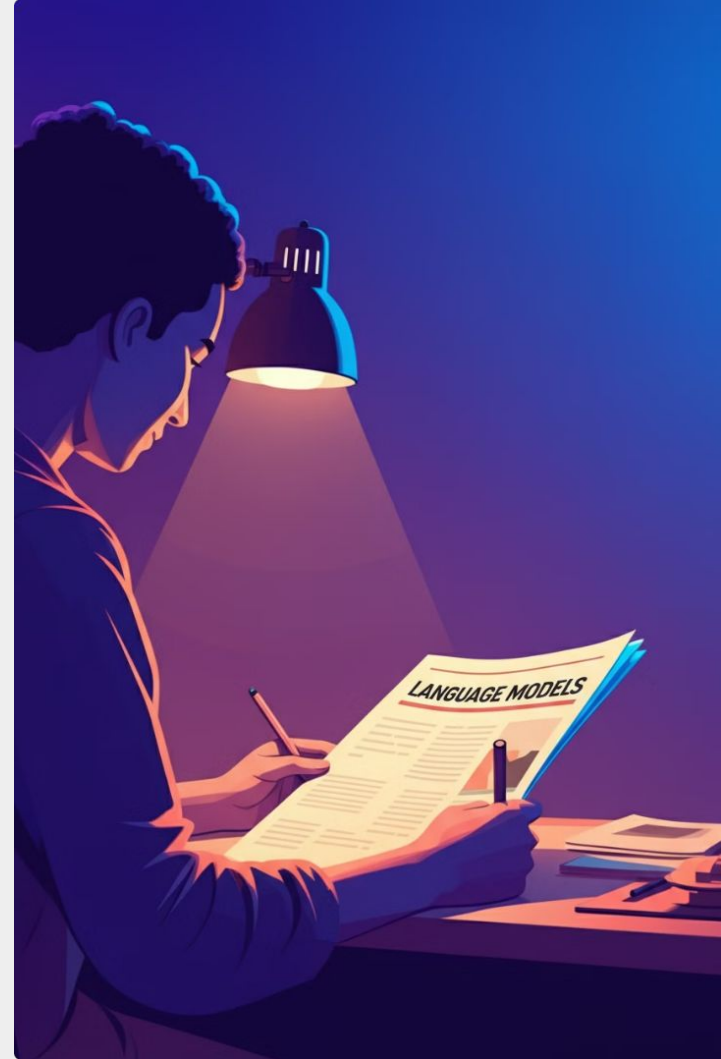
Princípio: Eliminar desperdício em todas as formas — funcionalidades não usadas, reuniões sem propósito, código sem validação.

Melhor para: Como filosofia transversal que potencializa qualquer framework ágil.



Próximos passos

- Eng. Software em Dimensões
 - Cap. 4 (seções 4.1–4.5):
Processos de software,
modelos, Scrum, Kanban
- Eng. Software Moderna
 - Cap. 2 — Processos Manifesto
Ágil, Scrum, Kanban



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
[viniciusgarcia.me](https://www.viniciusgarcia.me)

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Metodologias ágeis: Scrum, Kanban e a escolha do processo